The 14th IEEE International Conference on Dependable Systems, Services and Technologies, DESSERT'2024
11-13 October, 2024, Athens, Greece

2024 14th International Conference on Dependable Systems, Services and Technologies (DESSERT) | 979-8-3315-1093-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/DESSERT65323.2024.11122242

Extracting REST API Endpoints from Microservices Using LLM Agents

Oleh Chaplia
Department of Specialized Computer Systems
Lviv Polytechnic National University
Lviv, Ukraine
oleh.y.chaplia@lpnu.ua

Halyna Klym
Department of Specialized Computer Systems
Lviv Polytechnic National University
Lviv, Ukraine
halyna.i.klym@lpnu.ua

***Abstract*—This paper uses LLM agents to explore automated API endpoint extraction and OpenAPI documentation generation from Node.js microservices. The proposed system analyzes microservice source code, processes it, and extracts REST API endpoints with its data. As a result, the system provides a list of endpoints for the microservice and generates an OpenAPI file that can be used for further analysis. The system design and prototype were implemented, tested, and evaluated. Results show that the automated system reduces the time needed to prepare microservice documentation and helps the developers maintain it without managing routine tasks like writing and managing the documentation file or the comments in the source code. The LLM agentic approach provides a foundation for further automated source code analysis and generation, especially when big LLM models are used.**

Keywords—microservices, software architecture, cloud computing, artificial intelligence, LLM agents

I. Introduction

Large Language Models (LLMs) transform how modern software is designed and developed. LLMs cover many tasks, including text generation, code generation, summarization, information extraction, etc. Their ability to understand and generate human-like text allows developers to automate a wide range of previously manual, complex, or time-consuming processes. Software development relies heavily on managing and manipulating source code and text. Therefore, the capabilities of LLMs are beneficial when working with the source code.

For example, the development process is improved by using GitHub Copilot [1], automated writing of the source code, understanding code structures, identifying patterns, automated fixing and updating of issues in the code, and other routine tasks. Code generation is one example of the practical usage of modern LLMs [2].

Going further, new approaches combine multiple LLMs and other tools into a system of AI agents. AI agents can handle various tasks, especially while working in the environment [3]. These state-of-the-art multiagent solutions take all the benefits of cloud computing. Modern distributed systems consisting of many connected services use a microservice-based architecture [4]. Microservice is an architectural approach to building distributed systems, offering scalability, availability, modularity, and resilience benefits [5]. For example, Netflix has hundreds of microservices [6]. Nvidia NIM is a set of inference microservices implemented for production usage [7].

REST API is widely used for microservice communication [8]. Many routine tasks appear when writing REST API services or analyzing existing REST API services. These tasks include examining the code, writing documentation for the service, maintaining documentation, and testing. A proper software architecture and application design are needed to support the microservice codebase [9].

As the number of microservices grows, automating the extraction of REST API endpoints becomes essential to reducing human error, enhancing service discovery, and ensuring seamless communication between services. The challenges remain in managing the complexity of microservices, especially in identifying and extracting RESTful API endpoints from multiple services and understanding the interconnections.

Recent advancements in large language models (LLMs) have shown great potential in automating tasks traditionally requiring significant manual effort. Applying LLMs to extract REST API endpoints from microservices presents an innovative approach that leverages the capabilities of these models. LLMs can automatically parse source code, identify API definitions, and document the endpoints, simplifying the overall development process and improving system maintainability.

This paper proposes an automated approach to extracting REST APIs from the source code adapted for the Node.js microservice. The method aims to address the challenges associated with manual extraction by automating this process using LLMs. This approach reduces the manual time needed to create the documentation, reduces human errors, improves accuracy, and enhances the efficiency of managing microservice architectures. Furthermore, this approach can facilitate better integration within DevOps pipelines, promoting faster deployments and reducing the likelihood of integration failures caused by incomplete or inaccurate API documentation. This work contributes to the growing field of AI-assisted software engineering by demonstrating the practical application of LLMs in modern software architectures, specifically in the microservice ecosystem.

The rest of the paper is organized as follows. Section 2 presents challenges in managing the microservice REST API documentation. Section 3 outlines the research goals, methods, and questions for this research with the roadmap for addressing them. Section 4 discusses state-of-the-art approaches and tools. Section 5 provides the proposed approach. Then, Section 6 shows proof-of-concept implementation and evaluation. Section 7 provides the results. Finally, the paper's conclusion is presented in Section 8.

979-8-3315-1093-0/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:44:26 UTC from IEEE Xplore. Restrictions apply.

II. Challenges in Managing REST API Documentation

This section presents the challenges of API service documentation management. This section provides a complementary answer for the RQ1 and RQ2 regarding literature and drawbacks in addition to Section 4.

These challenges are the drawbacks that add more complexity to the manual and framework-automated documentation management for microservices. Identified challenges are used as a navigation space to find the places for improvements. As a result, we started to improve the automation process of the documentation extraction for the Node.js microservice. Using this approach, the system may be scaled further for other programming languages and frameworks.

Distributed cloud systems often use microservice architecture as a foundation. Microservices are designed for modularity and agility, allowing frequent updates, independent deployments, and rapid scaling.

Each service within a system may have asynchronous updates and deployments at any time. Therefore, this dynamic nature makes keeping REST API documentation current challenging. As APIs evolve—whether by adding new endpoints, parameter changes, or version deprecations—documentation must be continuously updated to reflect the most recent state of the service. Managing documentation becomes increasingly complex as services scale.

Microservices are often developed and maintained by different teams, which can result in inconsistent documentation practices across services. Accurate and comprehensive documentation is critical for developers integrating with REST APIs.

This inconsistency can complicate integration efforts, making it difficult for developers to understand how different APIs interact and which endpoints they should use. In some cases, developers may need to update the corresponding documentation to implement specific features or modifications, leading to discrepancies between the service's actual behavior and what is described. Over time, this can erode trust in the documentation, resulting in developers needing to dive into source code to understand how an API works.

Maintaining accurate documentation that represents the actual source code is essential. Microservice documentation can quickly become fragmented and unmanageable without a documentation framework that ensures consistency in format, terminology, and style.

Additionally, large-scale systems often require documentation for multiple environments—such as development, testing, and production—each of which may have slightly different configurations or behaviors. Managing these variations adds another layer of complexity. Automated documentation systems must be able to scale seamlessly across multiple environments while maintaining clarity and relevance for developers working in different contexts.

While automated tools can generate documentation based on code annotations or specifications, they often fail to capture the full scope of API functionality if not validated by a developer. Essential details like error handling, edge cases, or specific usage scenarios may be omitted or misrepresented. For instance, automated tools might generate endpoint descriptions without providing sufficient information about valid parameter ranges or response codes for different error conditions.

Even after REST API documentation is generated, it is crucial to test and validate its accuracy against the actual behavior of the API. This can be done through unit tests that simulate API requests and compare the responses to what is documented. However, setting up these validation mechanisms can be time-consuming, especially when dealing with many services and endpoints. Ensuring that all documented endpoints behave as expected requires continuous testing as part of the development lifecycle.

Summarizing the provided challenges, having consistent and up-to-date documentation for one microservice keeps the overall system well-defined. When each service has up-to-date documentation, multiple connections between services may be appropriately defined and validated.

III. Research goals and methods

The primary goal of this research is to improve the process of creating REST API documentation for microservices. To improve the documentation creation process, the manual documentation creation time should be reduced, the limitations of the framework tools should be addressed, and a flexible automated system should replace manual work.

An automated system that extracts REST API endpoints from Node.js microservices using Large Language Models was created to achieve the goal. This system addresses the challenges of maintaining up-to-date API documentation in distributed and dynamic microservice environments.

A set of research questions is defined for this work. At first, a thorough review of existing literature and approaches is needed (RQ1). Initially, a comprehensive review of existing techniques, methods, and tools for REST API documentation and microservice architectures is conducted. Frameworks and tools were examined. A review of the capabilities of LLMs and AI agents for automated software development and code analysis is conducted. These reviews provide the foundation for identifying drawbacks and limitations in current approaches.

The main challenges and drawbacks are determined based on the literature analysis and approaches (RQ2). Based on these insights, identified drawbacks are improved in the scope of the paper's goal.

Then, a system is designed and developed to use LLM agents to analyze source code and extract REST API endpoints (RQ3). The combination of static code analysis with the LLM-driven natural language processing (NLP) approach is used to automatically extract the API documentation from the microservice. The system architecture is designed to be an application running on a local machine. A Node.js microservice is created for testing purposes.

In summary, the system improves in response to the challenges and drawbacks (RQ4). The evaluation phase involves testing across several metrics, including lines of code, number of files, LLM processing time for each file, generation of OpenAPI file, the accuracy of API documentation, proof-of-concept performance, testing with various codebases, system overhead, and operational costs. These tests are conducted on a test Node.js microservice to ensure that the generated documentation aligns with expected

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:44:26 UTC from IEEE Xplore. Restrictions apply.

API behavior. The quality of the improvements is determined in the proof-of-concept testing and evaluation section.

Key insights are gathered regarding the strengths and limitations of using LLMs in this context. The research concludes with recommendations for further improvements and potential applications of LLM-powered documentation in larger and more complex microservice ecosystems.

IV. State-of-the-art Approaches and Tools

This section discusses modern techniques, approaches, documentation extraction, and generation tools. This discussion answers the RQ1. The pros and cons of these tools are discussed. Defined drawbacks are complementary to the drawbacks reviewed in Section 2. These drawbacks provide the answer to the RQ2. The analysis of the literature, existing approaches, and drawbacks gives a foundation and guidance for the following research questions (RQ3, RQ4).

Automated tools for REST API extraction exist, but they are often restricted to specific programming languages and their frameworks, such as Python, JavaScript, Java, Scala, C#, or Rust [10]. While effective within their respective ecosystems, these tools are limited to the framework's scope. They require a manual description of the APIs in the source code in the comments sections or as decorators. For instance, tools like OpenAPI specification [11] and SpringDoc [12] cater primarily to Java-based applications. FastAPI [13] and Pydantic [14] target Python environments.

Compared to Node.js microservices, there are few frameworks for the automated generation of API documentation. The JavaScript language specifics impact the way how the services are implemented. Most tools require manual addition of the API descriptions in the code, which is then gathered and processed. However, this step needs human input.

ApiDoc is a tool designed to create RESTful API documentation directly from comments within the source code [15]. It supports inline documentation for API methods, parameters, and responses and automatically generates a static web page displaying the API details in an organized manner. This fragmentation makes adopting a unified approach for automated documentation across heterogeneous microservice systems challenging.

TypeDoc is a documentation generator specifically for TypeScript projects [16]. It automatically creates API documentation directly from TypeScript source code and supports features like commenting and annotations.

RepoAgent, a tool designed to automate the generation and maintenance of code documentation for entire repositories using LLMs, features global structure analysis, documentation generation, and automatic updates to ensure consistency between code and documentation [17]. The framework is to improve team productivity by minimizing manual documentation tasks.

The authors of the SpeCrawler introduce a method to automate OpenAPI Specification (OAS) generation using a combination of rule-based scraping and LLMs [18]. The tool parses API documentation, extracting request-response pairs and generating base OAS files. It then enriches these with detailed parameters using LLM capabilities. SpeCrawler aims to streamline the integration of diverse APIs, reducing the manual effort required for creating OAS.

The RestGPT framework is designed to integrate LLMs with RESTful APIs to handle complex real-world instructions [19]. RestGPT utilizes a coarse-to-fine online planning mechanism for task decomposition and API selection, significantly improving flexibility and adaptability. It is evaluated using a custom benchmark, RestBench, demonstrating the model's ability to work with multiple APIs in real-world scenarios like the TMDB movie database and Spotify music player.

Respector is a tool that automates the generation of REST API specifications using static and symbolic program analysis [20]. It aims to reduce the time and errors involved in manually creating API specifications, which are essential for testing and usage. Evaluated on 15 real-world APIs, Respector showed high precision and recall in detecting methods, parameters, and responses, even identifying missing and inconsistent elements in developer-provided specifications. Respector also outperformed other approaches based on annotations or API invocations. However, in the context of this work, this tool uses static analysis instead of LLM agents.

V. Proposed Approach to Automated REST API Extraction

The proposed automated REST API extraction approach combines a static code analysis and large language models (LLMs) agents to generate comprehensive and accurate API documentation. This section provides the answer to the first part of RQ3 - the system architecture of the proposed approach. Each component of the system architecture is described, and connections are presented.

Fig. 1. presents a proposed application architecture. The architecture consists of the actual source code of the Node.js microservice and the application. The application contains several parts: a dependency extractor, a parser, storage for the source code data and metadata, LLM agents, and an orchestrator.

At first, the framework analyzes the source code using static analysis techniques. A dependency graph of the Node.js microservice is generated. The dependency graph in JSON format provides information about project files, their relations, connections between files, and their content with the metadata. Metadata contains information on whether the

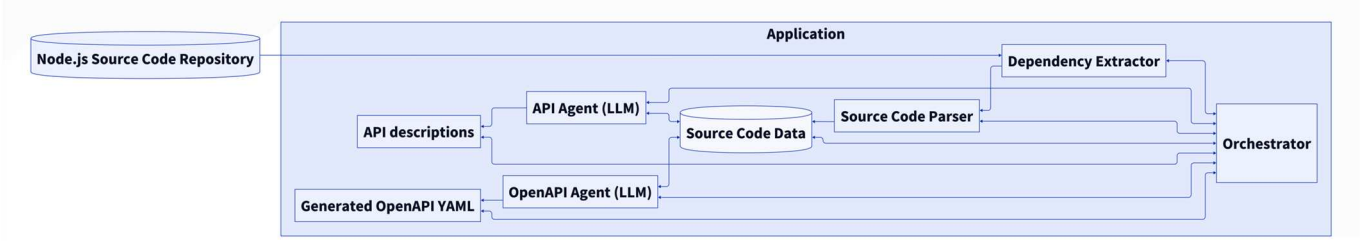


Fig. 1. The architecture of the proposed application.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:44:26 UTC from IEEE Xplore. Restrictions apply.

file is an NPM module, its relations to other files, the validity of the file, module type, and whether it can be resolved. Other metadata is present but not necessary for this research. A dependency graph as a PNG image helps to understand the project structure visually.

The source code parser parses the files extracted by the dependency graph. Then, the graph optimization process reduces the amount of information and presents the results in another JSON file. It tries to find any API endpoints and the information about them from the code, filtering any unnecessary information. A summarization and code analysis is executed in this step. It extracts REST API descriptions of the HTTP methods, URLs, expected body on input, possible response, and controllers. Then, this data is stored in JSON files for further local execution. A NoSQL or a graph database is recommended for storage in production systems.

The system contains two configurable agents. The first one, API Agent, is an LLM that receives the filtered source code and tries to extract, categorize, and summarize the found information about the endpoints. This information is formatted into a generic format, presented to the user, and stored in the storage or database. The purpose of the second agent is to receive extracted information about API endpoints and generate an OpenAPI file with the endpoints. OpenAPI file is a YAML file with the descriptions of all endpoints. This file can be completed and shared with any developer later. The system is evaluated by applying it to real-world microservices implemented in popular frameworks like Node.js.

VI. Proof-of-Concept Implementation and Evaluation

After the system architecture is created, a proof-of-concept application is developed. The implementation of the application is the second part of RQ3. The proposed approach was successfully implemented. Then, the implemented application is tested, and the results are evaluated.

The implementation of the prototype was designed, developed, and tested on a local machine. The local machine's characteristics are an Apple M1 processor and 16Gb RAM. OpenAI GPT-4o-mini model was used as an LLM for each agent [21]. According to the published model characteristics, GPT-4o provides better results but takes longer to process. GPT-4o-mini is faster and cheaper, but it may not be enough for some specialized tasks. It is entirely enough to extract the API information. For this research, GPT-4o-mini was selected. The application is self-contained and, therefore, can be easily transformed into a cloud-based solution. Python was used to implement the main code for the application. The Streamlit framework was used for the UI implementation [22].

For testing purposes, several projects were selected. Each project repository contains a Node.js server with the express.js framework. Each repository has various source code organization structures, file and folder names, various NPM packages, definitions of API routes, and endpoints. Each repository contains code-related JS files and additional files, which are filtered. However, the projects are similar because they use Node.js and express framework.

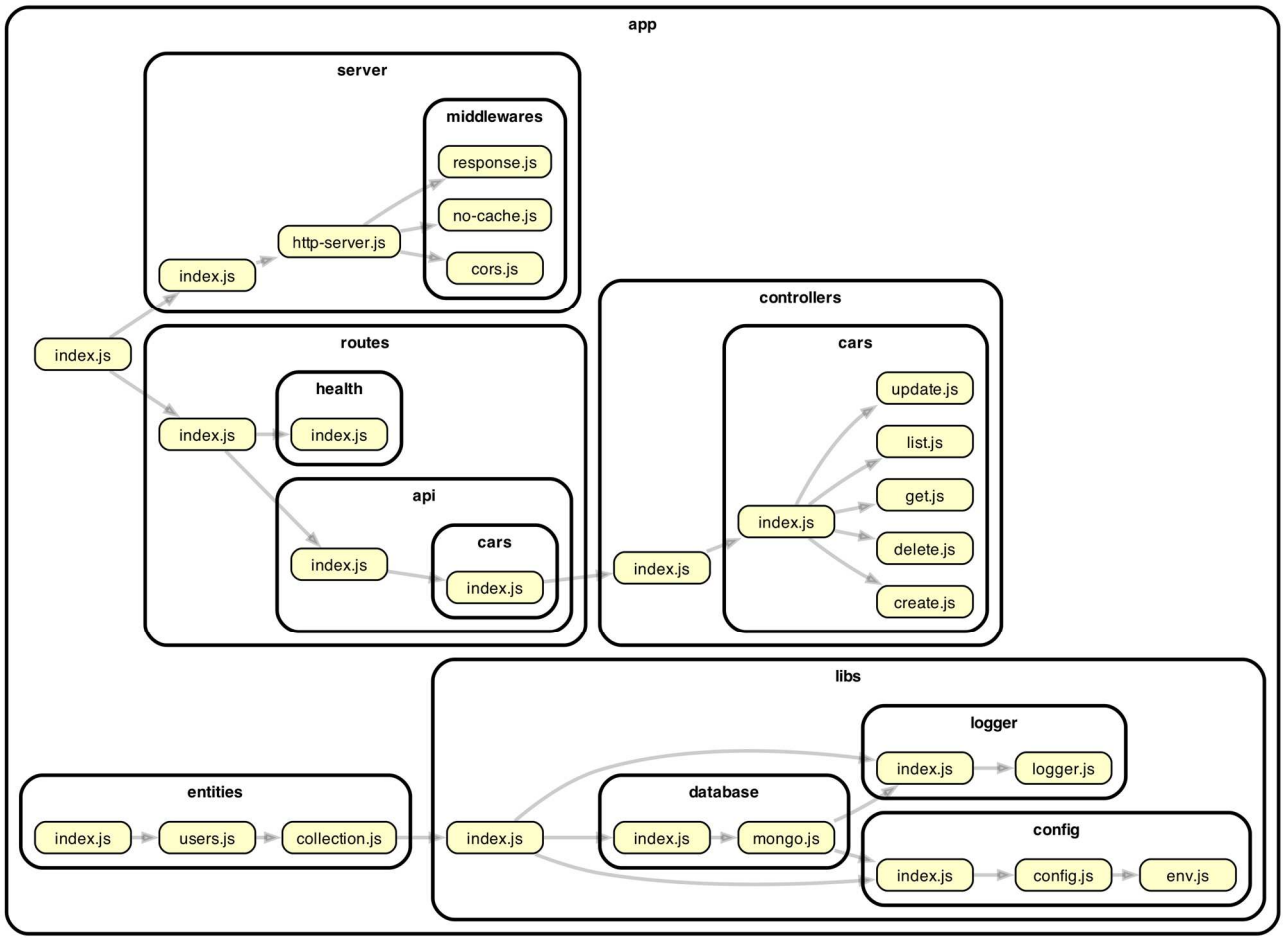


Fig. 2. Dependency graph for the test Node.js microservice.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:44:26 UTC from IEEE Xplore. Restrictions apply.

Table 1. Metrics gathered from the repositories processing.

Feature	Repository #1	Repository #2	Repository #3	Repository #4	Repository #5	Repository #6
Source code files count	28	39	43	50	76	89
Lines of code	474	1793	2254	2434	2671	3105
Graph processing time (seconds)	0.1	0.1	0.1	0.1	0.1	0.1
Min LLM processing time of one file (seconds)	0.51	0.68	0.55	0.48	0.66	0.59
Max LLM processing time of one file (seconds)	1.28	1.3	1.68	1.41	1.12	1.21
Sum of the LLM summarization time (seconds)	31.52	33.39	39.38	33.23	36.21	38.1
Time for OpenAPI file creation by LLM (seconds)	12.4	10.98	14.53	15.13	18.23	19.21
Manually found API endpoints	6	9	14	16	19	22
Automatically extracted API endpoints	5	9	14	16	18	21
Valid OpenAPI file?	Yes	Yes	Yes	Yes	Yes	Yes

The main idea of microservice architecture is to contain a small amount of code and domain-specific functionality. When a microservices system is scaled, the number of services is expected to be scaled. Therefore, having a flexible solution for one service may be applied to several services.

The Dependency Cruiser tool generates the dependency graphs for the test Node.js microservices [23]. Fig. 2. presents a dependency graph of one of the selected microservices for testing. On this graph, we can see where the endpoints dwell. Often, according to the best practices and patterns like MVC, API endpoint controllers are stored in separate folders named “api,” “apis,” “routes,” “controllers,” and “actions.”

The folder naming may help identify the APIs. Dependency graphs are generated for each test microservice. Based on these dependency graphs, the API endpoints are determined manually and compared to the generated endpoints.

In the current proof-of-concept implementation, JSON files were used to store the generated dependency graphs and additional data. Networkx library is used to process the graphs. A database, e.g., MongoDB or Neo4j, is recommended for production usage and scalability. Then, the CPU performance and RAM may be increased for the server that hosts the application. For this application, the main bottleneck is the LLM processing time.

VII. Results

A testing and validation process was conducted to evaluate the proposed cloud-based system using the LLM agent’s source code analysis. The system was tested on a local machine with an Apple M1 processor and 16Gb RAM. At first, ten various repositories were selected. Then, the prototype processed each repository, and an OpenAPI YAML file was generated as output. The gathered results

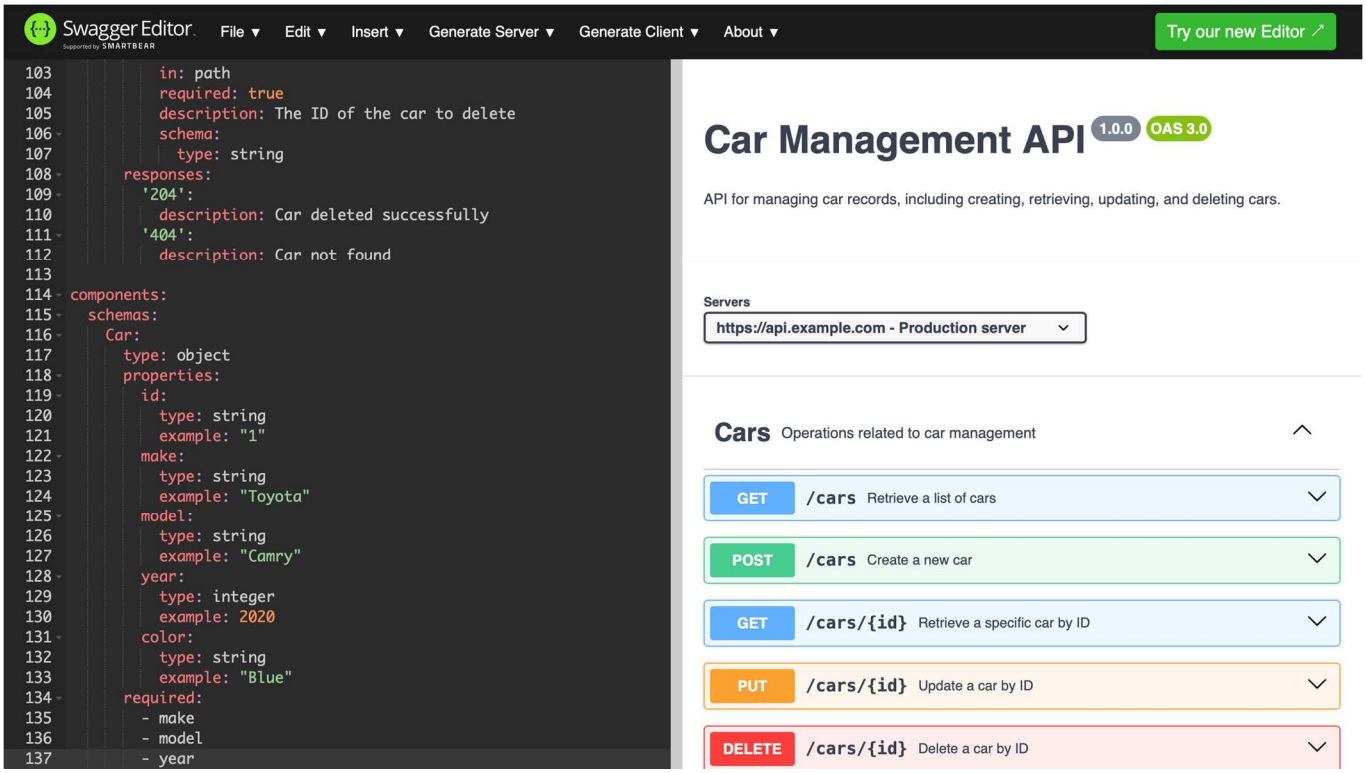


Fig. 3. Generated OpenAPI file from the repository source code.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:44:26 UTC from IEEE Xplore. Restrictions apply.

depict the answer to the RQ4 – improvements for the previously defined challenges and drawbacks.

The results of the six selected repositories are presented in Table 1. Based on the results, a processing pattern is visible. The local environment is complete enough to process the files and generate dependency graphs with a minimal latency of up to 0.5-1 seconds for each repository. Therefore, this latency was not considered. However, we can see that LLM processing varies up to 1.68, or even up to 2 seconds. For production usage, this network latency impacts the user experience and costs. More repositories were not added to the table because the values use the same pattern.

The number of manually found API endpoints and their descriptions are compared to those generated. An observation says that sometimes the model needs help understanding source code as an API endpoint when the code is distributed across the repository or when the file needs more information for extraction. Therefore, a dependency graph and file relations help understand the context of the complete source code.

The GPT-4o model costs $5.00 per million input tokens and $15.00 per million output tokens, while the more affordable GPT-4o Mini is priced at $0.15 per million input tokens and $0.60 per million output tokens. GPT-4o Mini is designed to offer high performance at a fraction of the cost, excelling in tasks such as reasoning, coding, and multimodal processing, making it an attractive option for developers looking for cost-effective solutions.

Table 1. also shows the lines of code for the test microservice. This project contains a minimal number of files and code. Knowledge of the lines of code helps predict the performance of bigger projects. LLM performance and inference cost depend on the amount of text provided. LLMs use tokens to measure the amount of input and output information and calculate the price. A larger amount of text is transformed into a larger amount of input tokens. Then, the inference time is larger because of the increased amount of input tokens. Also, input and output context limitations apply to various LLMs. Therefore, knowing the number of files and their lines of code can provide some measurement of the processing time and associated costs.

Fig. 3. Shows the OpenAPI file generated using the LLM agent. The file is in the YAML format. It can be visually presented using the OpenAPI editor. The result shows that the HTTP method, URL path, used status codes, summary, description, and tags are in the generated file. A proper description for each API endpoint has been added based on the LLM summarization.

We can observe several improvements compared to the existing solutions embedded in the frameworks. The first improvement is the possibility of LLMs allowing the summarization and writing of descriptions of the APIs based on the source code. For developers, this task is time-consuming. Therefore, an automated process of writing definitions and descriptions is helpful. The next improvement compared to the static code analyzers embedded in the frameworks that gather the API descriptions from the metadata and code decorators is that LLMs can parse various programming language codes. Therefore, there is no need to implement specialized parsers for each task. If needed, existing parsers may be used to prepare the code or even fine-tune the LLM for better results. The current proof-of-concept was tested on Node.js projects, but the approach is flexible.

However, several drawbacks appeared. LLM agents statically process a snapshot of the source code. Therefore, there is no information about the data from the runtime. Static information is extracted correctly, but some dynamic information is missing. For example, the expected HTTP request input body and dynamically generated responses may need to be added or improved. Also, some endpoints were missing when the declarations of these endpoints were embedded or located differently.

A dependency on external tools is another small challenge. Even when the local environment or CI/CD is well configured, security access to the shell execution from the Python code may be needed. It impacts the application's installation time. Another drawback is the latency. To process the source code, LLM agents need to process many files. Therefore, the network and processing latency are present. It varies from 40 seconds to more.

LLM hallucinations may appear, but the overall consistency of the results is sufficient to provide usable outcomes. A hallucination generates information that seems plausible or real but is not factually accurate or entirely fabricated. These hallucinations manifest as subtle inaccuracies and false details when complete information is not provided. Hallucinations are a known limitation of LLMs, and while they can be mitigated, they are not entirely avoidable, mainly when the model operates outside its core knowledge or training data.

Therefore, the time cost is reduced compared to manual documentation generation. Human validation is still needed, but validating and fixing errors in a generated file is more accessible than creating it manually.

VIII. Conclusion

First, a literature review and a review of the approaches were conducted (RQ1). Based on the results of the review, challenges and drawbacks were found (RQ2). A review of state-of-the-art techniques and tools was also conducted (RQ2). Then, the proposed approach was created to address the drawbacks, architected, and designed (RQ3). Based on the architecture, a proof-of-concept application was successfully implemented (RQ3). The results gathered after prototype testing were compared, evaluated, and analyzed (RQ4).

The tests cover several Node.js microservices. The system can extract information about API endpoints and its metadata. Then, the LLM agents process this information and store the database output. The LLM agent also generates an OpenAPI file.

In the future, it might be integrated with modern CI/CD pipelines, ensuring almost real-time updates to API documentation with each code deployment.

Combining LLM agents and code analysis provides a solid foundation for API extraction from Node.js microservices. This means that most routine tasks can be automated and simplified. Of course, human validation is needed, but most of the work is done automatically. AI-driven development helps automate many functions in the software development lifecycle.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:44:26 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] M. Wermelinger, "Using GitHub Copilot to Solve Simple Programming Problems," in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*, in SIGCSE 2023. New York, NY, USA: Association for Computing Machinery, Mar. 2023, pp. 172–178. doi: 10.1145/3545945.3569830.

[2] F. Lin, D. J. Kim, Tse-Husn, and Chen, "When LLM-based Code Generation Meets the Software Development Process," Mar. 23, 2024, *arXiv*: arXiv:2403.15852. doi: 10.48550/arXiv.2403.15852.

[3] T. Guo *et al.*, "Large Language Model based Multi-Agents: A Survey of Progress and Challenges," 2024, *arXiv*. doi: 10.48550/ARXIV.2402.01680.

[4] P. Ataei and D. Staegemann, "Application of microservices patterns to big data systems," *J Big Data*, vol. 10, no. 1, p. 56, May 2023, doi: 10.1186/s40537-023-00733-4.

[5] Y. Abgaz *et al.*, "Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review," *IIEEE Trans. Software Eng.*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023, doi: 10.1109/TSE.2023.3287297.

[6] Netflix Technology, "Migrating Critical Traffic At Scale with No Downtime — Part 1," Medium. Accessed: Feb. 12, 2024. [Online]. Available: https://netflixtechblog.com/migrating-critical-traffic-at-scale-with-no-downtime-part-1-ba1c7a1c7835

[7] "NIM for Developers," NVIDIA Developer. Accessed: Sep. 07, 2024. [Online]. Available: https://developer.nvidia.com/nim

[8] A. Neumann, N. Laranjeiro, and J. Bernardino, "An Analysis of Public REST Web Service APIs," *IEEE Trans. Serv. Comput.*, vol. 14, no. 4, pp. 957–970, Jul. 2021, doi: 10.1109/TSC.2018.2847344.

[9] O. Chaplia and H. Klym, "Designing a Node.js Project Architecture for RESTful Microservices," in *2023 IEEE 12th International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*, Dortmund, Germany: IEEE, Sep. 2023, pp. 808–811. doi: 10.1109/IDAACS58523.2023.10348681.

[10] S. Shan, M. Reyna, J. Xu, C. Skirvin, N. Patury, and R. Scott, "Cost Study of Test Automation Over Documentation for Microservices," *DEStech Transactions on Computer Science and Engineering*, no. csse, 2018, doi: 10.12783/dtcse/csse2018/24507.

[11] "OpenAPI Specification - Version 3.1.0 | Swagger." Accessed: Sep. 07, 2024. [Online]. Available: https://swagger.io/specification/

[12] O. 3 L. for spring-boot B. B. N. LAHSEN and L. for O. 3 with spring-boot B. B. N. LAHSEN, "OpenAPI 3 Library for spring-boot," OpenAPI 3 Library for spring-boot. Accessed: Sep. 07, 2024. [Online]. Available: http://springdoc.org/

[13] "FastAPI." Accessed: Sep. 07, 2024. [Online]. Available: https://fastapi.tiangolo.com/

[14] "Welcome to Pydantic - Pydantic." Accessed: Sep. 07, 2024. [Online]. Available: https://docs.pydantic.dev/latest/

[15] "apiDoc - Inline Documentation for RESTful web APIs." Accessed: Sep. 07, 2024. [Online]. Available: https://apidocjs.com/

[16] "TypeDoc," TypeDoc Documentation Generator. Accessed: Sep. 07, 2024. [Online]. Available: https://typedoc.org/

[17] Q. Luo *et al.*, "RepoAgent: An LLM-Powered Open-Source Framework for Repository-level Code Documentation Generation," Feb. 26, 2024, *arXiv*: arXiv:2402.16667. doi: 10.48550/arXiv.2402.16667.

[18] K. Lazar *et al.*, "SpeCrawler: Generating OpenAPI Specifications from API Documentation Using Large Language Models," Feb. 18, 2024, *arXiv*: arXiv:2402.11625. doi: 10.48550/arXiv.2402.11625.

[19] Y. Song *et al.*, "RestGPT: Connecting Large Language Models with Real-World RESTful APIs," Aug. 26, 2023, *arXiv*: arXiv:2306.06624. Accessed: Jun. 04, 2024. [Online]. Available: http://arxiv.org/abs/2306.06624

[20] R. Huang, M. Motwani, I. Martinez, and A. Orso, "Generating REST API Specifications through Static Analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, in ICSE '24. New York, NY, USA: Association for Computing Machinery, Apr. 2024, pp. 1–13. doi: 10.1145/3597503.3639137.

[21] "OpenAI Platform." Accessed: Sep. 07, 2024. [Online]. Available: https://platform.openai.com

[22] "Streamlit • A faster way to build and share data apps." Accessed: Sep. 07, 2024. [Online]. Available: https://streamlit.io/

[23] S. Verweij, *sverweij/dependency-cruiser*. (Sep. 06, 2024). JavaScript. Accessed: Sep. 07, 2024. [Online]. Available: https://github.com/sverweij/dependency-cruiser

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:44:26 UTC from IEEE Xplore. Restrictions apply.